Автоматизированное Рабочее Место

Программно-аппаратный комплекс для автоматизации работы конкретного сотрудника

Made with GAMMA

Что такое АРМ?

АРМ представляет собой комплексное решение, которое интегрирует программное обеспечение, аппаратные компоненты и периферийные устройства для оптимизации работы сотрудника.

Ролевая модель

Настройка интерфейса и функциональности под конкретную роль

Оптимизация задач

Фокус на специфических рабочих процессах

Интеграция периферии

Работа со сканерами, ККМ, терминалами

Надежность

Стабильная работа в производственной среде

Made with GAMMA

Где применяются АРМ?

АРМ кассира

Работа с контрольно-кассовой техникой, сканером штрихкодов, системами эквайринга

АРМ кладовщика

Приемка товаров, инвентаризация, работа с терминалами сбора данных

АРМ менеджера

CRM-системы, обработка заказов, печать документов

АРМ бухгалтера

Электронный документооборот, формирование отчетности

Made with GAMMA

Технологический стек

Современный стек технологий для создания надежных desktop-приложений

Технология	Назначение
C# / .NET	Язык программирования и платформа
WPF	Desktop-интерфейс и визуализация
XAML	Декларативная разметка пользовательского интерфейса
MVVM	Архитектурный паттерн разделения ответственностей
Entity Framework Core	ORM для работы с базами данных
SQL Server / PostgreSQL	Реляционная база данных

Made with GAMMA

Почему именно WPF?

Windows Presentation Foundation обеспечивает гибкость и мощь для создания профессиональных интерфейсов

Data Binding

Автоматическая синхронизация данных между UI и бизнес-логикой

Разделение XAML/C#

Четкое разделение дизайна интерфейса и программной логики

Архитектура MVVM

Тестируемость, поддерживаемость и масштабируемость кода

Стили и шаблоны

Гибкая настройка визуального стиля без изменения кода

Богатая экосистема

Готовые компоненты, темы и сторонние библиотеки

Made with GAMMA

Многослойная архитектура

Presentation

WPF-приложение, Views, ViewModels, пользовательский интерфейс

Business (Core)

Сервисы, модели данных, бизнес-интерфейсы, логика приложения

Data Access

Репозитории, DbContext, работа с источниками данных

Database

SQL Server, PostgreSQL, хранение и управление данными

Важно: Зависимости направлены только сверху вниз через абстракции

Made with GAMMA

Model-View-ViewModel

Архитектурный паттерн для разделения ответственностей в приложении

1

View

XAML-файлы, элементы управления, визуальное отображение

2

ViewModel

Состояние UI, команды, логика представления, уведомления

3

Model

Данные, бизнес-сущности, правила валидации

Made with GAMMA

Ключевые механизмы MVVM

Data Binding

```
<TextBox Text="{Binding UserName}" />
```

Двусторонняя связь между UI-элементом и свойством ViewModel

INotifyPropertyChanged

```
public string UserName
{
  get => _userName;
  set
  {
    _userName = value;
    OnPropertyChanged();
  }
}
```

Уведомление об изменении свойства для обновления интерфейса

Commands

```
<Button Command="{Binding SaveCommand}" />
```

Привязка действий пользователя к методам в ViewModel

Dependency Injection

```
public MainViewModel(
  IDataService service)
{
  _service = service;
}
```

Внедрение зависимостей через конструктор

Made with GAMMA

Внедрение зависимостей

Проблема жесткой связи

```
var service = new DataService();
// Невозможно заменить реализацию
// Трудно тестировать
```

Решение через DI

```
public MainViewModel(
  IDataService service)
{
  _service = service;
}
// Гибкость и тестируемость
```

Слабая связанность

Компоненты не знают о конкретных реализациях друг друга

Легкое тестирование

Можно подменить реальные сервисы на моки и заглушки

Гибкая замена

Изменение реализаций без переписывания кода

Made with GAMMA

Практическая часть

Создадим структуру проекта и настроим основу для работы

01

Создание решения

Инициализация ARM_Solution с тремя проектами

02

Настройка зависимостей

Конфигурация ссылок между ARM.Client, ARM.Core и ARM.Data

03

Установка NuGet

Добавление пакетов для DI-контейнера и других зависимостей

04

Реализация ViewModel

Создание базового MainViewModel с INotifyPropertyChanged

05

Настройка DI

Конфигурация контейнера зависимостей и регистрация сервисов

06

Запуск и проверка

Сборка и тестирование работы приложения

Made with GAMMA